

RAG 程式說明

[參考程式](#)

RAG Pipeline

- **Embedding 向量化**: 將文本轉為數學座標。
- **Chunking 文本切分**: 處理長文章的必要步驟。
- **FAISS 索引構建**: 實現極速的語義搜尋。
- **Prompt 注入**: 將搜尋結果餵給 LLM。
- **Generation 生成**: 模型根據參考資料回答問題

[外部知識庫]



向量搜尋



大語言模型回答

Step 1: Environment Setup

首先安裝核心向量運算庫 **FAISS**。注意版本號以確保相容性。

```
# 安裝 FAISS CPU 版本 (Colab 環境適用)
!pip install faiss-cpu==1.11.0.post1

# 建立數據集目錄並下載 Wikipedia 內容 (NCKU 範例)
!mkdir -p demo_dataset && cd demo_dataset && \
wget -nc https://raw.githubusercontent.com/yasaisen/LLMTutorial/main/demo_dataset/ncku_wikipedia_251008
```

Step 2: Hugging Face 驗證

存取 Google 的 Gemma 模型需要通過官方授權。

至 Hugging Face 申請 Access Token。

Token 權限需設為 Read。

這一步驟決定了模型能否成功下載。

```
# 填入您的 HF Token  
!huggingface-cli login --token hf_XXX...
```

General Methods Define — 載入 LLM 模型

create_model() 載入語言模型

```
def create_model(  
    lm_model_name = "google/gemma-3-4b-it",  
    device = 'cuda' if torch.cuda.is_available() else 'cpu',  
):  
    tokenizer = AutoTokenizer.from_pretrained(lm_model_name)  
    model = AutoModelForCausalLM.from_pretrained(  
        lm_model_name,  
        device_map="auto", # 自動分配 GPU 顯存  
    ).eval()  
    return model, tokenizer
```

General Methods Define — Prompt 格式設定

lm_template() 建立 Chat Prompt

```
def lm_template(text: str, system_prompt: str = "You are a helpful assistant."):
    return [
        {"role": "system", "content": [{"type": "text", "text": system_prompt}]},
        {"role": "user", "content": [{"type": "text", "text": text}]}
    ]
```

角色	功能描述
System	定義模型的「人格」。例如可以改為 "You are a professional expert on the history of Chengdu University"。
User	使用者輸入的實際問題(在 RAG 中會包含檢索到的參考內容)。

General Methods Define — 檢查輸入長度是否超過模型限制

避免 Prompt 超過模型長度限制

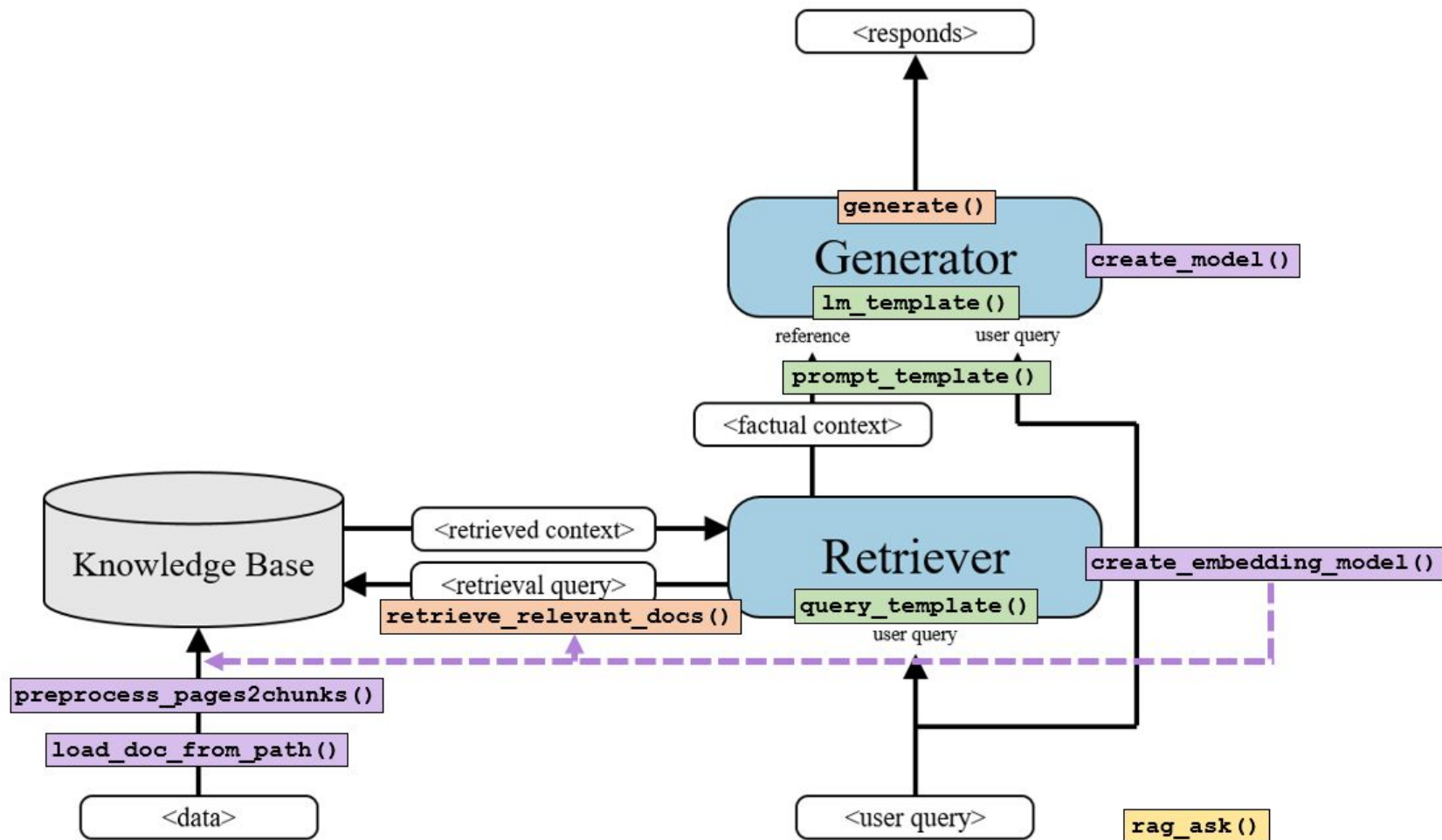
```
@torch.inference_mode()
def generate(prompt, tokenizer, model, max_new_tokens: int = 256):
    # 將文字轉換為 Token 並移動到顯卡 (CUDA)
    inputs = tokenizer.apply_chat_template(prompt, tokenize=True, return_tensors="pt")

    # 長度檢查：防止輸入超過模型承載上限
    max_len = int(model.config.max_position_embeddings)
    if input_len > max_len:
        raise ValueError("Input length exceeds maximum allowed length.")
```

General Methods Define — generate() 產生回答

RAG 的參考資料如果太多，容易導致輸入長度爆炸

```
generation = model.generate(  
    **inputs,  
    max_new_tokens = max_new_tokens,  
    do_sample = True,  
    temperature = temperature,
```



RAG Methods Define — create_embedding_model()

把文件與使用者的問題轉換成數學向量

```
def create_embedding_model(  
    emb_model_name: str = "all-MiniLM-L6-v2",  
    device = 'cuda' if torch.cuda.is_available() else 'cpu',  
):  
    embedding_model = SentenceTransformer(  
        emb_model_name  
    ).to(device).eval()  
  
    return embedding_model
```

RAG Methods Define — 定義資料結構、提示詞模板

定義資料結構 (Data Structures)

DocChunk 類別：系統中最小的文本單位與特徵載體

lookupQuery 類別：將使用者的輸入字串(question)封裝成一個物件

```
@dataclass
class DocChunk:
    idx: int
    content: str
    embedding: np.ndarray = None
    score = None

@dataclass
class lookupQuery:
    question: str
```

實作提示詞模板 (Prompt Templates)

query_template：負責提取用於向量檢索的查詢字串

prompt_template：將檢索到的外部知識與問題

組裝成標準化的 Prompt，並嚴格約束模型的輸出格式

```
def query_template(
    query,
):
    return f"""
{query.question}
"""

def prompt_template(
    context,
    query,
):
    return f"""
References:
{context}
Question:
{query.question}
Do not use markdown syntax to answer and put the answer after "Answer:"
"""
```

RAG Methods Define — preprocess_pages2chunks()

建置知識庫的資料前處理

將文字標準化並附上 ID

第一階段：

清理原始資料

第二階段：

賦予身分 ID 並封裝成物件

有了 `idx`, 當 FAISS 向量庫找到最相似的向量時, 我們才能依據這個 ID 找回它原本對應的文字內容 (`content`)

```
def preprocess_pages2chunks(
    pages_list,
):
    chunked_list = []
    for page in pages_list:
        chunked_list.append({
            'text': page['text'].strip(),
        })

    idx = 0
    all_chunks = []
    for chunked in chunked_list:
        doc = DocChunk(
            idx=idx,
            content=chunked['text'],
        )
        all_chunks += [doc]
        idx += 1

    return all_chunks
```

RAG Methods Define — load_doc_from_path ()

建置知識庫：

把外部的原始資料讀進來，切割整理後，

轉換成電腦能快速搜尋的「向量索引」

讀取原始檔案 (Data Loading)

文本切塊與整理 (Chunking)

向量化 (Embedding)

建立 FAISS 向量索引庫 (Indexing)

資料綁定與回傳

```
def load_doc_from_path(
    documents_path: str,
    embedding_model,
):
    pages_list = []
    with open(documents_path, 'r', encoding='utf-8') as file:
        for line in file:
            line = line.strip()
            if line:
                data = json.loads(line)
                pages_list.append(data)

    chunk_list = preprocess_pages2chunks(
        pages_list=pages_list
    )
    contents = [doc.content for doc in chunk_list]

    embeddings = embedding_model.encode(contents)
    index = faiss.IndexFlatIP(embeddings.shape[1]) # Create FAISS index
    faiss.normalize_L2(embeddings)
    index.add(embeddings.astype(np.float32))

    # Save documents
    for doc, embedding in zip(chunk_list, embeddings):
        doc.embedding = embedding

    return chunk_list, index
```

RAG Methods Define — load_doc_from_path ()

建立 FAISS 向量索引庫 (Indexing)

1 建立空索引庫 (程式碼中的 `index` 代表 FAISS 搜尋引擎本體)

2 向量正規化 `normalize` 後, 向量長度會變成 1,
搭配 Inner Product 可等效於 Cosine Similarity

3 將 embedding 向量加入 FAISS index

建立可供 Similarity Search 的向量索引

(IndexFlatIP 使用 Inner Product 作為相似度計算方式)

補充說明 (搭配第 15 頁)

- FAISS 僅負責儲存向量與計算相似度, 不會自動檢查或合併重複內容。
- 不同 chunk 的文字內容相同, 可能產生相同或極度接近的 embedding, 並以不同 idx 存入 index。
- 本系統目前以 doc.idx 作為去重依據, 因此無法去除 embedding 相同但 idx 不同的結果。

```
def load_doc_from_path(
    documents_path: str,
    embedding_model,
):
    pages_list = []
    with open(documents_path, 'r', encoding='utf-8') as file:
        for line in file:
            line = line.strip()
            if line:
                data = json.loads(line)
                pages_list.append(data)

    chunk_list = preprocess_pages2chunks(
        pages_list=pages_list
    )
    contents = [doc.content for doc in chunk_list]

    embeddings = embedding_model.encode(contents)
    index = faiss.IndexFlatIP(embeddings.shape[1]) # Create FAISS index
    faiss.normalize_L2(embeddings)
    index.add(embeddings.astype(np.float32))

    # Save documents
    for doc, embedding in zip(chunk_list, embeddings):
        doc.embedding = embedding

    return chunk_list, index
```

RAG Methods Define — retrieve_relevant_docs()

檢索

把使用者的問題也變成 embedding

FAISS 查詢最高分(k個)的文章編號

用編號換回真實文章

剔除重複的文章片段 (DocChunk)

補充備註:「這裡的『剔除重複』是透過比對 ID (`idx`), 來防禦進階檢索(如多重查詢合併)時不小心重複拉取到『同一個實體物件』, 藉此節省 AI 的運算 Token。」

```
def retrieve_relevant_docs(
    search_query: str,
    embedding_model,
    index,
    chunk_list,
    top_k: int = 3
) -> List[DocChunk]:

    relevant_docs = []
    query_embedding = embedding_model.encode([search_query])
    faiss.normalize_L2(query_embedding)
    scores, indices = index.search(query_embedding.astype(np.float32), top_k)

    for score, idx in zip(scores[0], indices[0]):
        if idx < len(chunk_list):
            doc = chunk_list[idx]
            relevant_docs.append(doc)

    seen = set()
    unique_data = []
    for doc in relevant_docs:
        if doc.idx not in seen:
            seen.add(doc.idx)
            unique_data.append(doc)

    return unique_data
```

RAG Methods Define — rag_ask()

接受提問 → 尋找知識 → 產出答案

將使用者問題(純文字)包裝成具有結構的資料物件

執行知識檢索 (Retriever)

將檢索結果組裝成參考資料

建構提示詞 (Prompt Engineering)

呼叫模型生成答案 (Generator)

打包回傳結果

```
def rag_ask(
    user_query: str,
    model,
    tokenizer,
    embedding_model,
    index,
    chunk_list,
    top_k: int = 3,
    max_new_tokens: int = 256,
    temperature: float = 1,
) -> str:
    query = lookupQuery(
        question=user_query,
    )
    search_query = query_template(
        query=query,
    )
    relevant_docs = retrieve_relevant_docs(
        search_query=search_query,
        embedding_model=embedding_model,
        index=index,
        chunk_list=chunk_list,
        top_k=top_k,
    )

    context = ""
    for i, doc in enumerate(relevant_docs):
        context += f"References {i+1}:{doc.content}\n"

    text = prompt_template(
        context=context,
        query=query,
    )
    prompt = lm_template(
        text=text
    )
    response = generate(
        prompt=prompt,
        tokenizer=tokenizer,
        model=model,
        max_new_tokens=max_new_tokens,
        temperature=temperature,
    )

    return {
        'response': response,
        'prompt': prompt,
        'relevant_docs': relevant_docs,
    }
```

RAG Methods Define — rag_ask()

接受提問 → 尋找知識 → 產出答案

將使用者問題(純文字)包裝成具有結構的資料物件

執行知識檢索 (Retriever)

```
def rag_ask(  
    user_query: str,  
    model,  
    tokenizer,  
    embedding_model,  
    index,  
    chunk_list,  
    top_k: int = 3,  
    max_new_tokens: int = 256,  
    temperature: float = 1,  
) -> str:  
    query = lookupQuery(  
        question=user_query,  
    )  
    search_query = query_template(  
        query=query,  
    )  
    relevant_docs = retrieve_relevant_docs(  
        search_query=search_query,  
        embedding_model=embedding_model,  
        index=index,  
        chunk_list=chunk_list,  
        top_k=top_k,  
    )
```

RAG Methods Define — rag_ask()

將檢索結果組裝成參考資料

建構提示詞 (Prompt Engineering)

呼叫模型生成答案 (Generator)

打包回傳結果

```
context = ""
for i, doc in enumerate(relevant_docs):
    context += f"References {i+1}:{doc.content}\n"

text = prompt_template(
    context=context,
    query=query,
)
prompt = lm_template(
    text=text
)
response = generate(
    prompt=prompt,
    tokenizer=tokenizer,
    model=model,
    max_new_tokens=max_new_tokens,
    temperature=temperature,
)

return {
    'response': response,
    'prompt': prompt,
    'relevant_docs': relevant_docs,
}
```

Create Models & load documents

Generator LLM

Embedding Model

建置Knowledge Base

```
model, tokenizer = create_model(  
    lm_model_name="google/gemma-3-1b-it" # use 1b for colab..  
)  
embedding_model = create_embedding_model(  
    emb_model_name="all-MiniLM-L6-v2"  
)  
  
documents_path = './demo_dataset/ncku_wikipedia_2510080406.jsonl'  
  
chunk_list, index = load_doc_from_path(  
    documents_path=documents_path,  
    embedding_model=embedding_model,  
)
```

Cases Test

測試問題 一題一題

```
test_cases = [  
    "When was National Cheng Kung University established?",  
    "Who is the current president of National Cheng Kung University?",  
    "Where is National Cheng Kung University located?",  
    "What is the English translation of NCKU's motto?",  
]
```

放進rag_ask()

最後只取 ['response']

```
print("Start Demo!\n")  
  
for i, query in enumerate(test_cases, 1):  
    print(f"Test Case ({i}) {'=' * 50}")  
    print(f"user input: {query}")  
  
    response = rag_ask(  
        user_query=query,  
        model=model,  
        tokenizer=tokenizer,  
        embedding_model=embedding_model,  
        index=index,  
        chunk_list=chunk_list,  
    )['response']  
    print(f"Model response: {response}\n{'=' * 64}\n")  
  
print("\nDemo Completed!")
```